MCAST Institute for the Creative Arts

AI Use Declaration Form

This form must be completed and submitted with every assignment, as per the department's Academic Integrity and AI Policy. Transparency is essential to proving your creative ownership and learning process.

Part 1: Student & Assignment Details

Student Name: Yan Cesare

ID Number: 0142204L

Unit: API systems

Task: 1

Date: 27/03/26

Part 2: Declaration of AI Use

Please select one of the following options:

☐ **Option A: No AI Used:** I declare that I did not use any generative AI tools in the conception, development, or creation of this assignment. The submitted work is entirely my own.

☒ **Option B: AI Used:** I declare that I did use AI tools to assist in the creation of this assignment. I have provided a transparent and accurate account of its use in Part 3 below and have submitted all required process work in support of my final submission.

Part 3: Description of How AI Was Used

(Please tick all that apply and provide a brief, specific example for each. For an explanation of each refer to the AI Use Policy Manual)

☐ Ideation & Brainstorming

☐ Research & Summarization

☐ Text Generation & Refinement (Writing/Scripting)

☐ Image/Asset Generation:

☒ Other (Please specify):

Used if for rewriting my words o be more professional

☒ I confirm that I have submitted all required process and development materials alongside my final work, as outlined in the AI Policy.

☒ I confirm that the information provided on this form is a complete and accurate representation of my use of AI tools for this assignment. I have read and understood the Academic Integrity and AI Policy, and I am aware that failing to declare AI use, or misrepresenting its use, constitutes **plagiarism** and will result in disciplinary action.

Print to PDF and include in your submission once completed.

Part 1: RESTful APIs

Brief Explanation of Application

Infotainment for Maltese Citizens. This application will be focusing around an interactive map where a user will be able to zoom around the map to find interactive elements like bubbles which will indicate there is news in the locality. The target audience will be **25 and under specifically for students or those interested in journalism.** Journalists will be able to post on this application under a Journalism Membership explained later, there will also be a premium version of this application without ads *and can get discounts possibly.* This application will also allow users to comment under articles, bookmark certain articles as well. Similar to social media but with verification systems and actual credibility.

Core Principles

A REST API is an application programming interface (API) that follows the design principles of the Representational State Transfer (REST) architectural style. This style is commonly used to connect distributed hypermedia systems. REST APIs are also known as RESTful APIs or RESTful web APIs.

REST Design Principles

At its most basic level, an API is a mechanism that allows an application or service to access a resource within another application, service, or database. The system requesting the resource is known as the client, while the system providing the resource is the server.

Unlike APIs such as SOAP or XML-RPC, which enforce strict frameworks, REST APIs offer flexibility. Developers can build them using any programming language and support multiple data formats. However, REST APIs must comply with six architectural constraints.

Uniform Interface

All API requests for a specific resource should follow a consistent format, regardless of where the request originates. A REST API ensures that each piece of data, such as a user's name or email address, is associated with a single Uniform Resource Identifier (URI). Resources should be structured efficiently, containing all necessary information without being excessively large.

Client-Server Decoupling

In REST API architecture, the client and server operate independently of each other. The client only needs to know the URI of the requested resource and cannot directly interact with the server beyond making HTTP requests. Similarly, the server should not modify the client, except by returning the requested data through HTTP responses.

Statelessness

REST APIs are stateless, meaning that every request must include all the information required to process it. The server does not store any client-related session data. Each request is treated as an independent interaction, ensuring scalability and reliability.

Cacheability

Whenever possible, responses should be cacheable on either the client or server side. Server responses must indicate whether caching is permitted for a given resource. This improves performance on the client side and enhances scalability on the server side.

What is an API Call?

An API call is a request sent from a client, such as a web browser, mobile application, or another server, to an API endpoint in order to perform an operation. The API processes the request and returns a response, usually containing data or confirmation that the operation was successful.

A useful analogy is an ATM transaction:

1. The user (client) requests money, representing the API call.
2. The ATM (API) communicates with the bank's system (server).
3. The bank verifies the account and processes the request.
4. The ATM returns the cash, which represents the API response.

APIs enable applications to reuse functionality such as authentication, payment processing, or data retrieval without rebuilding these features from scratch.

Key Components of an API Call

Every API call includes several essential elements:

- HTTP Method: Specifies the operation (GET, POST, PUT, PATCH, DELETE)
- Endpoint URL: The address of the requested resource
- Headers: Metadata such as authentication details and content type
- Request Body: Data sent with POST, PUT, or PATCH requests
- Query Parameters: Optional filters appended to the URL

How API Calls Work

When an API call is made, the following sequence occurs:

1. The client initiates an HTTP request to an API endpoint
2. The request travels the network to the server
3. The server processes the request and validates input
4. The server interacts with a database if required
5. A structured response, often in JSON format, is generated
6. The response is returned to the client for processing

HTTP Methods in APIs

HTTP (Hypertext Transfer Protocol) methods define standardized actions that clients use to interact with server resources. In RESTful APIs, these methods correspond to CRUD operations (Create, Read, Update, Delete).

Key Methods

GET	Retrieves data from the server
POST	Creates a new resource or submits data
PUT	Updates or replaces an existing resource
DELETE	Removes a resource

These methods form the foundation of API interactions, ensuring predictable and efficient communication. For example, retrieving a webpage involves a GET request, while submitting a form typically involves a POST request.

Basic request codes

200 OK	**The request succeeded**
204 NO CONTENT	**The request was successful, but there is no additional content to send.**
301 MOVED PERMANENTLY	**The URL of the requested resource has been changed permanently.**
400 BAD REQUEST	**The server cannot process the request due to client error (e.g., malformed syntax).**
404 NOT FOUND	**The server cannot find the requested resource.**
500 INTERNAL SERVER ERROR	**The server encountered an unexpected situation.**

Parametrization in REST APIs

Parametrization refers to configuring requests by passing data that influences the response. In programming, parameters affect the output of functions, and similarly, in REST APIs, parameters influence the server's response.

Since REST APIs are stateless, each request must include all necessary information. Roy Fielding described this by stating that each request must contain all the information required for the server to understand it independently of previous requests.

Parameters can be included in several parts of an HTTP request, including query strings, request bodies, and headers. Each method has specific use cases and rules.

Parameter Placement and Usage

The simplest approach is placing all parameters in the request body, which is common in some legacy APIs. In such cases, endpoints often use POST requests exclusively, with all parameters included in the body. However, this is generally considered poor design and can be avoided with proper planning.

Standard HTTP headers provide predefined fields for common parameters:

- Accept Header: Specifies the desired response format (e.g., JSON or XML)
- Cache-Control Header: Controls caching behavior
- Authorization Header: Handles authentication

After evaluating standard headers, developers must decide whether to use custom headers or query strings for additional parameters.

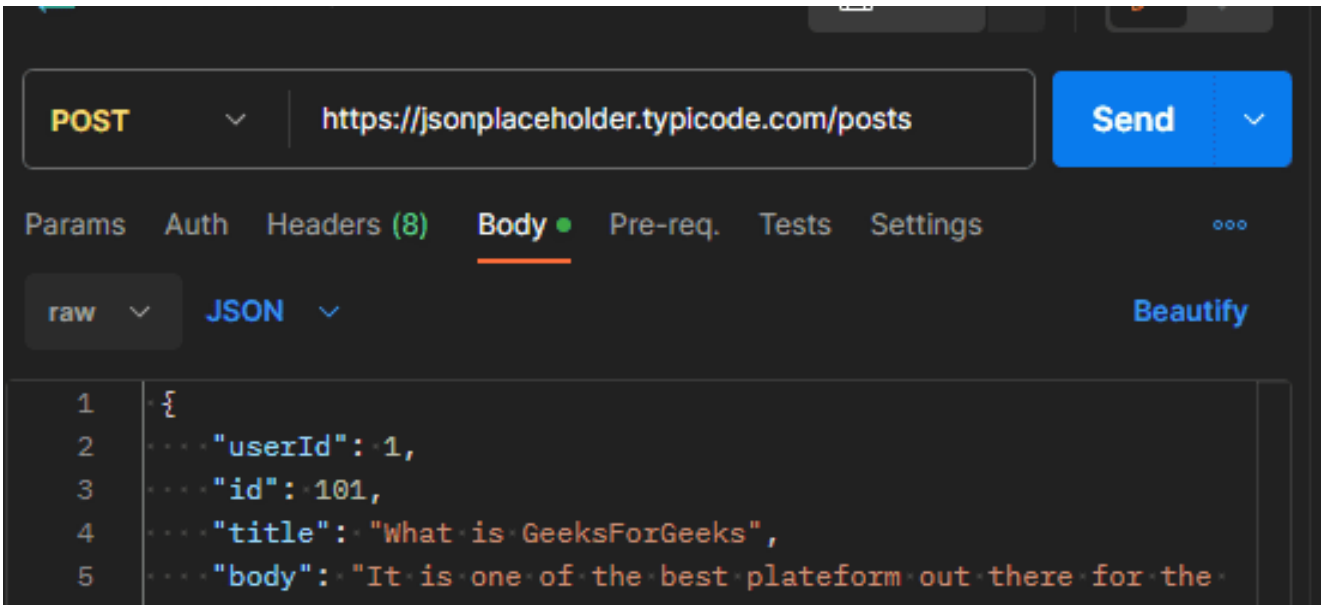
When to Use Query Strings

Query strings are typically used for filtering data and are especially useful for searching and pagination. Historically, they were used to send queries to servers, and later adapted for web forms using GET requests.

They can also support additional functionality, such as modifying responses. For example, adding a parameter like ?withComments may include related data within a single response.

Choosing between query strings and headers often depends on developer experience and usability. Headers are ideal for parameters that remain consistent across requests, such as authentication tokens. Query strings are better suited for dynamic, endpoint-specific parameters like filters.

E.g used in postman



How request bodies are structured

Depending on the particular API and the kind of data being delivered, request bodies can be organised in a number of ways. Three typical kinds are as follows:

Unprocessed. Although this can theoretically include any type of text, it typically contains JSON or XML, such as "firstName\Pied\lastName\Piper"

x-www-form-urlencoded. Simple text values in a query string, such as key1=value1&key2=value2, are sent using this. Every character is url encoded; for example, spaces are substituted with %20.

Binary. Images, movies, audio, and other non-text media can be attached using this method.

Multipart forms and GraphQL are two more variants. The server's expectations determine the proper format, and the API documentation should always specify how to submit data in a fashion that will be appropriately handled

Overview

Based on RESTful API design principles, the following endpoints are proposed for the Maltese infotainment application. These endpoints follow key REST constraints such as statelessness, uniform interface, and proper use of HTTP methods. Each endpoint demonstrates how query parameters and request bodies are used to manage data efficiently, along with appropriate HTTP status codes for response handling.

1. News Retrieval (Interactive Map)

Description:
This endpoint is used to retrieve news articles based on geographical coordinates. It supports the interactive map feature by displaying location-based news bubbles and allows filtering by category and verification status.

Field	**Details**
Endpoint	/api/news
HTTP Method	GET
Purpose	Retrieve news based on location
Query Parameters	lat (float), lng (float), radius (int), category (string), verifiedStatus (string)
Request Body	Not required
Response Codes	200 OK (success), 400 Bad Request (invalid parameters), 404 Not Found (no results)

2. Create News Article (Journalists)

Description:
This endpoint allows users with a journalism membership to upload news articles. The article is initially marked as “InProcess” until verified by the system.

Field	Details
Endpoint	/api/news
HTTP Method	POST
Purpose	Create a new article
Query Parameters	None
Request Body	title, content, category, location (lat, lng), mediaUrl, authorId
Response Codes	201 Created (success), 400 Bad Request (missing fields), 401 Unauthorized (not logged in), 403 Forbidden (not a journalist)

3. Update Verification Status

Description:
Used by verifiers to update the credibility status of an article. This supports the platform's verification system.

Field	Details
Endpoint	/api/news/{id}/verify
HTTP Method	PATCH
Purpose	Update article verification
Query Parameters	None
Request Body	status, verifiedBy, verificationType
Response Codes	200 OK (updated), 400 Bad Request, 401 Unauthorized, 403 Forbidden (not verifier), 404 Not Found

4. Filter News

Description:
Allows users to filter news articles based on preferences such as category, sensitivity, and date range.

Field	Details
Endpoint	/api/news/filter
HTTP Method	GET
Purpose	Filter news articles
Query Parameters	category, excludeSensitive (boolean), dateFrom, dateTo
Request Body	Not required
Response Codes	200 OK, 400 Bad Request

5. Bookmark Article

Description:
Enables users to save articles for later viewing.

Field	Details
Endpoint	/api/users/{userId}/bookmarks
HTTP Method	POST
Purpose	Save article
Query Parameters	None
Request Body	articleId
Response Codes	201 Created, 400 Bad Request, 401 Unauthorized

6. Retrieve Bookmarks

Description:
Fetches all bookmarked articles for a specific user.

Field	Details
Endpoint	/api/users/{userId}/bookmarks
HTTP Method	GET
Purpose	Get bookmarked articles
Query Parameters	None
Request Body	Not required
Response Codes	200 OK, 401 Unauthorized, 404 Not Found

7. Add Comment to Article

Description:
Allows users to post comments under news articles, supporting interaction similar to social media platforms.

Field	Details
Endpoint	/api/news/{id}/comments
HTTP Method	POST
Purpose	Add comment
Query Parameters	None
Request Body	userId, comment, timestamp
Response Codes	201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found

8. Report Suspicious Content

Description:
Provides users with the ability to report articles that may contain misinformation or false content.

Field	Details
Endpoint	/api/news/{id}/report
HTTP Method	POST
Purpose	Report article
Query Parameters	None
Request Body	userId, reason, details
Response Codes	201 Created, 400 Bad Request, 401 Unauthorized

9. User Authentication

Register User

Description:
Allows new users to create an account to access the application.

Field	Details
Endpoint	/api/auth/register
HTTP Method	POST
Purpose	Register new user
Request Body	name, email, password, role
Response Codes	201 Created, 400 Bad Request (invalid input), 409 Conflict (email exists)

Login User

Description:
Authenticates users and allows access to the system.

Field	Details
Endpoint	/api/auth/login
HTTP Method	POST
Purpose	Authenticate user
Request Body	email, password
Response Codes	200 OK, 400 Bad Request, 401 Unauthorized

10. Journalism Membership Application

Description:
Allows users to apply for journalism membership by submitting verification documents.

Field	Details
Endpoint	/api/journalism/apply
HTTP Method	POST
Purpose	Apply for membership
Request Body	userId, idCard, portfolio, qualifications, studentCard, specialisation
Response Codes	201 Created, 400 Bad Request, 401 Unauthorized

11. Heatmap Data (Trending News)

Description:
Provides aggregated data showing areas with high news activity for visual heatmap representation.

Field	Details
Endpoint	/api/news/heatmap
HTTP Method	GET
Purpose	Retrieve trending areas
Query Parameters	region
Request Body	Not required
Response Codes	200 OK, 400 Bad Request

12. Protest Events

Description:
Retrieves upcoming protest-related events to support awareness features within the application.

Field	Details
Endpoint	/api/events/protests
HTTP Method	GET
Purpose	Retrieve protest events
Query Parameters	date, location
Request Body	Not required
Response Codes	200 OK, 400 Bad Request, 404 Not Found

Part 2: Authorisation

OAuth 2.0 Overview

OAuth 2.0 is the industry-standard protocol for authorization. It is designed to simplify client development while providing secure authorization flows for web, mobile, desktop, and other applications.

The OAuth 2.0 framework allows third-party applications to gain limited access to HTTP services, either on behalf of a user or independently.

OAuth Access Tokens

An OAuth access token is a string used by a client to make requests to a resource server. These tokens can vary in format depending on implementation.

Access tokens are categorized as:

- Bearer Tokens: Usable by any party that possesses them
- Sender-Constrained Tokens: Require proof of possession, such as a private key

Important security properties include:

- Tokens must not be interpreted by the client
- Tokens do not expose user identity
- Tokens are only used to access resource servers, not as identity tokens

OAuth Client Types

OAuth defines two types of clients:

- Confidential Clients: Applications capable of securely storing credentials
- Public Clients: Applications that cannot securely store secrets, such as browser-based or mobile apps

How OAuth 2.0 Will Be Implemented in the Infotainment API

With respect to the Maltese infotainment application, OAuth 2.0 will be implemented to ensure that access to certain features like posting articles, commenting, bookmarking, and restricted access is carried out securely.

Authentication and Token Issuance

When a user tries to access the application through the /api/auth/login endpoint, authentication will be carried out by validating their credentials. Once authentication is complete, an access token will be issued by the server.

Implementation Details:

- The access token will be sent in the response
- Tokens will be implemented using the Bearer Token scheme
- The access token will be required for all API calls

Example Header Usage:

Authorization: Bearer <access_token>

This will ensure that only authenticated users are able to access certain resources.

Securing API Endpoints

All sensitive API endpoints will require a valid OAuth access token. These include:

- Creating a new news article – /api/news
- Commenting on an article
- Bookmarking an article
- Reporting an article
- Applying for journalism membership

Implementation Approach:

- The access token will be sent in the header for each API call
- The access token will be validated by the server
- If not provided or invalid, access will be denied

Response Codes:

- 401 Unauthorized – Token not provided or invalid
- 403 Forbidden – Provided token is invalid or access is denied

Role-Based Access Control (RBAC)

The OAuth 2.0 protocol will be extended to support different user roles in the application.

Defined Roles:

- **Regular Users** - Can view, comment, bookmark, and report content
- **Journalists** - Can create and manage articles
- **Verifiers** - Can approve and verify content

Implementation:

- The user roles will be embedded in the access token payload.
- The server will validate roles before granting access to certain endpoints.

Example:

- Only users with the role of a journalist can access the POST /api/news endpoint.
- Only verifiers can access the /api/news/{id}/verify endpoint.

Token Validation and Security

To ensure maximum security, all incoming tokens will be validated by the API before access is granted.

Steps to Validate Tokens:

- Check the integrity and signature of the token.
- Check if the token has not expired.
- Check user identity and roles.

Additional Security Measures:

- Tokens will be issued with an expiration time to prevent misuse.
- HTTPS will be enforced for all API requests.

Support for Public and Confidential Clients

The application will support public and confidential OAuth 2.0 clients.

Public Clients:

- Mobile and web apps for general users.
- Cannot securely store client credentials.
- Rely on access tokens for authentication.

Confidential Clients:

- Backend services like admin or verification systems.
- Can securely store client credentials.
- Used for high-level operations like verification.

Integration with Application Features

- OAuth 2.0 will be used to support the application's features:
- Interactive Map: Public access, but authenticated access for personalized information
- Commenting & Bookmarking: Only authenticated users allowed
- Journalism Dashboard: Only authenticated journalist accounts allowed
- Verification System: Only authenticated verifier accounts allowed
- Premium Features: Based on authenticated user status

Part 3: Security Considerations

OWASP Top 10 (2025)

The OWASP Top 10 is a widely recognized standard that identifies the most critical security risks in web applications. It reflects a global consensus among security professionals.

Top Risks Include:

The **OWASP Top 10 API Security Risks** highlight the most critical security risks to APIs.

- **API1:2023 – Broken Object Level Authorization**: This threat occurs when an API does not properly restrict access to certain objects based on user permissions. This means unauthorized users can access or modify data they should not be able to, potentially exposing sensitive information or allowing malicious actions.
- **API2:2023 – Broken Authentication**: Broken authentication happens when the methods used to verify the identity of users are flawed. This can allow attackers to take over user accounts, gain unauthorized access, and perform actions as if they were legitimate users.
- **API3:2023 – Broken Object Property Level Authorization**: This issue happens when APIs do not properly check permissions at the property level within objects. It may allow attackers to gain unauthorized access or to modify specific data fields within an object, exposing sensitive data or enabling malicious actions.
- **API4:2023 – Unrestricted Resource Consumption**: APIs need resources like network bandwidth, CPU, memory, and storage to function. When APIs do not have adequate limits on resource consumption, attackers can overwhelm the API by making excessive requests, leading to denial of service or increased operational costs.
- **API5:2023 – Broken Function Level Authorization**: Broken function level authorization means that the API does not properly restrict access to certain functions or actions based on user roles. This allows users to perform actions they should not be able to, such as accessing administrative functions.
- **API6:2023 – Unrestricted Access to Sensitive Business Flows**: APIs may expose business processes, such as purchasing items or posting comments, without adequately controlling how these functions can be abused. Attackers may automate misuse of these functionalities to harm the business.
- **API7:2023 – Server Side Request Forgery (SSRF)**: SSRF occurs when an API fetches resources from another server without properly validating the user-provided URL. Attackers can exploit this to make the API server send requests to unintended locations, bypassing firewalls or accessing internal systems.
- **API8:2023 – Security Misconfiguration**: Security misconfiguration involves having incorrect or incomplete settings in an API, making it vulnerable to attacks. This can happen

due to default settings, missing patches, or misconfigured security controls that lead to potential exploits.

- **API9:2023 – Improper Inventory Management**: This risk occurs when APIs expose more endpoints than necessary or lack proper documentation and version management. Attackers can exploit outdated, deprecated, or unsecured endpoints and API versions that developers might have forgotten about.
- **API10:2023 – Unsafe Consumption of APIs**: Developers may trust data from third-party APIs too much and not apply the same security standards they use for user inputs. Attackers can exploit vulnerabilities in these third-party services to compromise the API indirectly.

Overview

From the OWASP API Security Top 10 (2023), two key risks have been selected due to their relevance to the infotainment application:

API3: Broken Object Property Level Authorization
API10: Unsafe Consumption of APIs

These risks are particularly important given the platform's reliance on user-generated content, role-based permissions (users, journalists, verifiers), and potential integration with third-party services.

API3: Broken Object Property Level Authorization

Description

This vulnerability occurs when an API does not properly enforce access control at the property level within objects. As a result, users may be able to view or modify sensitive fields that should be restricted.

Application Risk

In the infotainment application, this risk is highly relevant due to the presence of structured objects such as news articles, user profiles, and verification data.

For example:

A normal user could attempt to modify the verificationStatus of an article
A user could change their role to "journalist" or "verifier"
Sensitive fields such as internal verification notes could be exposed

Mitigation Strategy

To prevent this vulnerability, the following measures will be implemented:

Strict Server-Side Validation:
The API will only accept and process specific allowed fields from the request body. Any unexpected or restricted fields will be ignored or rejected.
Field-Level Access Control:
Different user roles will have access to different properties:

Users → basic fields (comments, profile info)
Journalists → article content fields
Verifiers → verification-related fields

DTOs (Data Transfer Objects):
Separate request and response models will be used to control which fields can be sent and received, ensuring sensitive properties are never exposed.
Immutable Sensitive Fields:
Fields such as role, verificationStatus, and verifiedBy will only be modifiable through protected endpoints (e.g., verifier-only routes).

Example Implementation

For the endpoint:
PATCH /api/news/{id}

A journalist can update:

```
{
  "title": "Updated Title",
  "content": "Updated content"
}
```

If the request includes:

```
{
  "verificationStatus": "Verified"
}
```

→ The API will reject or ignore this field unless the user is a verifier.

API10: Unsafe Consumption of APIs

Description

This vulnerability occurs when an application trusts data from external or third-party APIs without proper validation or security checks.

Application Risk

The infotainment application may rely on third-party services such as:

Map services (for the interactive map)
Media/image hosting
External news or event feeds

If these external APIs return malicious or unvalidated data, it could:

Inject harmful content into the application
Display incorrect or misleading information
Compromise system integrity

Mitigation Strategy

To reduce this risk, the following measures will be implemented:

Input Validation and Sanitisation:
All data received from third-party APIs will be validated and sanitised before being used or displayed.
Whitelisting Trusted Sources:
Only approved and trusted API providers will be used (e.g., verified map or media services).
Error Handling and Fallbacks:
The system will not rely entirely on third-party responses. If an API fails or behaves unexpectedly, fallback mechanisms will be used.
Data Filtering:
Only required fields from external APIs will be extracted, avoiding unnecessary exposure to potentially unsafe data.
Timeouts and Rate Limits:
External API calls will be controlled to prevent abuse or system overload.

Example Implementation

If a third-party API provides map or event data:

The API will:

- Validate coordinates and text fields
- Strip any scripts or unsafe content
- Only store necessary attributes (e.g., location, title)

Any unexpected or malformed response will be rejected before reaching the client.

References

https://blog.postman.com/what-is-an-api-call

Confidential and Public Clients - OAuth 2.0 (2026) *Oauth.net*. Available at: https://oauth.net/2/client-types/ (Accessed: 27 March 2026).

HTTP Methods in APIs: GET, POST, PUT, DELETE Explained - API7.ai (2025) *HTTP Methods in APIs: GET, POST, PUT, DELETE Explained - API7.ai*. Available at: https://api7.ai/learning-center/api-101/http-methods-in-apis.

IBM (2021) *REST APIs*, *Ibm.com*. Available at: https://www.ibm.com/think/topics/rest-apis.

Ploesser, K. (2019) *REST API Design Best Practices for Parameter and Query String Usage*, *REST API Design Best Practices for Parameter and Query String Usage | Moesif Blog*. Available at: https://www.moesif.com/blog/technical/api-design/REST-API-Design-Best-Practices-for-Parameters-and-Query-String-Usage/.

What Are the OWASP Top 10 API Security Risks? | Akamai (2025) *Akamai*. Available at: https://www.akamai.com/glossary/what-is-owasp-api-top-10.

What is an API Request Body? | API Connector (2022) *Mixed Analytics*. Available at: https://mixedanalytics.com/knowledge-base/request-bodies-explained/.